

Lab6 Register Allocation 26sp

ddl 2026/6/24 23:59

1. Lab Description

Finish register allocation in your tiger compiler.

Related files for this lab are:

- `src/tiger/liveness/*.*` Files related to liveness analysis
- `src/tiger/regalloc/*.*` Files related to register allocation

To finish this lab, you will only need to finish the following modules: { **liveness analysis** } { **register allocation** }, you can modify the following files to finish your design.

Code block

```
1  .— src/tiger/parse/tiger.y
2  |— src/tiger/lex/tiger.lex
3  |— src/tiger/lex/scanner.h
4  |— src/tiger/absyn/absyn.*
5  |— src/tiger/semant/semant.*
6  |— src/tiger/escape/escape.*
7  |— src/tiger/frame/frame.h
8  |— src/tiger/frame/temp.*
9  |— src/tiger/frame/x64frame.*
10 |— src/tiger/translate/translate.*
11 |— src/tiger/translate/tree.*
12 |— src/tiger/canon/canon.*
13 |— src/tiger/codegen/assem.*
14 |— src/tiger/codegen/codegen.*
15 |— src/tiger/liveness/flowgraph.*
16 |— src/tiger/liveness/liveness.*
17 |— src/tiger/regalloc/color.*
18 |— src/tiger/regalloc/regalloc.*
19 |— src/tiger/util/graph.h
20 |— src/tiger/util/table.h
```

In your generated code, you should do what you can to **avoid unnecessary stack access** (push and pop).

This means using **escape analysis** to identify variables that can be kept in registers before the register allocation phase.



Notice: Before you start this lab, you should carefully read chapters 10,11 of the textbook.

2. Implementation Hints

1. Now that you've completed register allocation for your Tiger compiler, the generated assembly can be linked with `gcc` to produce a real executable that runs on actual hardware. The testing process for this lab includes the following steps:
 - a. Build the `tiger-compiler` executable.
 - b. Use `tiger-compiler` to compile a Tiger source file into assembly.
 - c. Link the generated assembly with the provided runtime library using `gcc`.
 - d. Run the resulting executable and compare its output with the expected reference output.
2. During register allocation, follow these main steps:
 - a. Construct the control flow graph (CFG).
 - b. perform liveness analysis on the CFG to determine where each temporary is live.
 - c. Build the interference graph, where an edge indicates two temporaries cannot share a register, according to the result of liveness analysis.
 - d. Apply graph coloring to assign physical registers to temporaries. If there are not enough registers, mark some for spilling, rewrite the program to include load/store instructions, and repeat the analysis.

3. Environment

Merge the code from lab5-2, and update the branch lab6 from gitee.

4. Local Grade Test

The lab environment contains a grading script named `grade.sh`. You can use it to evaluate your code, and that's how we grade your code, too. If you pass all the tests, the script will print a successful hint, otherwise, it will output some error messages. You can execute the script with the following commands.

Code block

```
1 ~/tiger-lab6 > bash ./scripts/grade.sh lab6
```

```
2  Pass bsearch
3  Pass dec2bin
4  Pass merge/test1
5  Pass merge/test2
6  Pass merge/test3
7  Pass merge/test4
8  Pass prime
9  Pass qsort
10 Pass queens
11 Pass tbi
12 Pass test_array
13 Pass test_patch
14 Pass tfact
15 Pass tfo
16 Pass tif
17 Pass tiffn
18 Pass tlink
19 Pass trec
20 Pass tree
21 Pass twhi
22 [^_^]: Pass
23 LAB6 SCORE: 100
```

5. Submission Instructions

1. Use `make ziplab6` command to generate the zip file. Do not change any files other than those included in the `make ziplab6` command.
2. Hand in the zip file to Gradescope website.